

STUDENT PROJECT REPORT

Project Title: Student Management System

Student Name: Suryansh Somvanshi

Registration Number: 26BAI10382

Course: B.Tech CSE (AIML)

Programming Language Used: Python

1. Project Overview

This project is a simple command-line terminal script written in Python to keep track of student details without using any database software or GUI. Everything runs inside the terminal.

The program allows adding new student records, searching for students by their registration number, updating existing details, deleting records, and printing all stored student entries in a clear table layout.

Main Goals & Features

- **Simple CLI Interface:** Easy to run from terminal or command prompt.
- **Modular Code:** Code is divided into clean separate script files.
- **Persistent Data:** Automatically saves student records to records.txt.
- **No Dependencies:** Runs strictly on standard Python with no extra library installations required.

2. Project Files & Structure

The code is split across distinct script files to keep the logic simple and easy to understand:

- **main.py:** The entry point script that runs the program loop and calls actions based on user selection.
- **menu.py:** Prints the interactive menu choices (1-6) on the terminal screen.
- **records.py:** Handles adding, searching, updating, and deleting student details.
- **file_handler.py:** Reads data from and writes data back to records.txt.
- **display.py:** Formats and prints individual records and complete table views.
- **config.py:** Holds global configuration settings like file paths.
- **records.txt:** Plain text file storing data persistently.

3. How the Script Works (Step-by-Step)

1. **Program Start:** Running python main.py calls file_handler.py to check and read records.txt.
2. **Data Loading:** Existing student entries are parsed into memory.
3. **Menu Loop:** menu.py displays options (1. Add, 2. Search, 3. Update, 4. Delete, 5. Show All, 6. Exit).
4. **Action Processing:** Based on choice (1 to 6), records.py performs the action on the student data.
5. **Auto Saving:** When records are added, modified, or deleted, file_handler.py rewrites records.txt.
6. **Program Exit:** Selecting option 6 breaks the main execution loop and exits.

4. Code Analysis & Module Explanation

Configuration (config.py)

Defines global constants like FILENAME = "records.txt" so file settings can be updated from one place.

Operations (records.py)

- add_records(): Takes registration number, name, hostel block, state, email, and contact, checking for duplicate registration numbers before saving.
- search_student(): Looks up and displays student info using registration number.
- update_student(): Updates information for an existing registration number.
- delete_student(): Removes a record from memory and saves changes.

File Handling (file_handler.py)

- load_records(): Opens records.txt, reads lines using .readlines(), strips whitespace, and splits attributes using " | " delimiter.
- save_records(): Formats records back into pipe-delimited strings and saves them using file .write().

Output Display (display.py)

Contains show_student() and show_all() to print neat tabular headers and aligned records on the console.

5. Future Scope

- Adding GUI using Tkinter or PyQt.
- Adding regex-based input validation for emails and phone numbers.

- Converting file storage into SQLite or MySQL database for larger scale.